

Buscacompis

Julián Camilo Hortúa Franco
Matemáticas Discretas I
Universidad Nacional de Colombia
Bogotá, Colombia
26 de Julio de 2026

Resumen—Formar grupos de estudio es una buena forma de profundizar y afianzar conocimientos, pero encontrar con quién estudiar suele ser una barrera difícil de superar, especialmente para estudiantes que están empezando la carrera o que no conocen a nadie en una materia. Buscacompis es un proyecto que aborda este problema calculando qué tan compatibles son dos estudiantes a partir de las materias que cursan, su horario disponible y sus hobbies, y usando esa información para sugerir posibles compañeros y grupos de estudio. La solución modela los atributos de cada estudiante como conjuntos, define la compatibilidad como una relación binaria simétrica calculada mediante el índice de Jaccard sobre esos conjuntos, y representa al conjunto completo de estudiantes como un grafo no dirigido donde los nodos representan a los estudiantes y las aristas indican una compatibilidad entre dos estudiantes mayor o igual a un umbral de compatibilidad específico. Con este grafo se implementaron una forma de hacer recomendaciones individuales de posibles compañeros, una forma de detección de grupos de estudio mediante cliques, y una breve verificación de propiedades formales de la relación como simetría y transitividad. El programa se hizo en Python, primero validando la lógica en un Jupyter Notebook y luego construyendo una interfaz interactiva en Streamlit. Las pruebas confirmaron que el cálculo de compatibilidad y la construcción del grafo funcionan correctamente, aunque se identificó una limitación relacionada con la normalización de los datos de entrada. El resultado más importante es que, con un umbral de 0.5, el modelo genera un grafo equilibrado que conecta a la mayoría de los estudiantes sin saturar las recomendaciones, validando que conjuntos, relaciones y grafos son herramientas efectivas para resolver este problema.

Index Terms—matemáticas discretas, conjuntos, grafos, relaciones, índice de Jaccard

I. INTRODUCCIÓN

Estudiar en grupo conlleva beneficios que no se logran con un estudio individual: permite resolver dudas más rápido, entender temas desde distintos puntos de vista, repartir la carga de trabajo y mantener la motivación cuando el semestre se pone pesado. Sin embargo, en la práctica muchos estudiantes, sobre todo quienes están empezando la carrera o entran a materias donde no conocen a nadie, tienen dificultad para formar estos grupos, se necesita compartir clase con alguien, coincidir en horarios disponibles y, en muchos casos, hay una barrera simplemente para acercarse y proponerle a un desconocido estudiar juntos. Vale la pena resolver este problema porque el punto de entrada a un grupo de estudio saber con quién empezar suele ser la parte más difícil, no el trabajo en grupo en sí. Reducir esa barrera inicial puede traducirse en mejores resultados académicos y en menos estudiantes trabajando de forma aislada por no saber a quién acercarse. A partir de esta

idea surge Buscacompis, un proyecto que busca facilitar ese primer acercamiento calculando de forma automática qué tan compatibles son dos estudiantes y, a partir de esos cálculos, agrupando al conjunto completo de estudiantes en una red donde se pueden identificar grupos de estudio. El objetivo general de este proyecto es construir una forma sencilla y automática de identificar qué tan compatibles son dos estudiantes a partir de lo que tienen en común, y usar esa información para sugerir posibles grupos de estudio. El alcance de este trabajo fue el siguiente: se definió qué significa que dos estudiantes sean compatibles usando conjuntos, relaciones y el índice de Jaccard; se implementó el cálculo de compatibilidad y la construcción del grafo resultante en Python sobre un conjunto de estudiantes simulados; y se analizaron propiedades del grafo (componentes conexas, grado de nodo) para verificar que el modelo efectivamente agrupa a estudiantes afines. No se desarrolló una interfaz de usuario ni se probó con datos reales de estudiantes, quedando esto como trabajo futuro.

II. FUNDAMENTOS DE MATEMÁTICAS DISCRETAS

Conjuntos: Un conjunto es una colección bien definida de objetos, sobre la cual se pueden definir operaciones como la unión ($\cup$), que agrupa todos los elementos presentes en al menos uno de los conjuntos, y la intersección ($\cap$), que agrupa únicamente los elementos presentes en ambos. En este proyecto cada estudiante se representa mediante tres conjuntos: el conjunto de materias que cursa, el conjunto de franjas horarias en las que está disponible, y el conjunto de hobbies o pasatiempos que practica. Modelar estos atributos como conjuntos y no como listas o texto libre permite usar directamente las operaciones de unión e intersección para medir qué tanto comparten dos estudiantes, que es la base de todo el cálculo de compatibilidad del proyecto.

Relaciones: Una relación binaria sobre un conjunto S es un subconjunto de pares ordenados de $S \times S$ que indica qué elementos de S están vinculados entre sí según algún criterio; entre sus propiedades más relevantes están la simetría (si a se relaciona con b , entonces b se relaciona con a) y la transitividad (si a se relaciona con b y b con c , entonces a se relaciona con c). En este proyecto, la compatibilidad entre dos estudiantes se define como una relación binaria sobre el conjunto de estudiantes: dos estudiantes están relacionados si su nivel de compatibilidad calculado con Jaccard supera un umbral definido. Esta relación es la que decide si dos personas terminan conectadas en el grafo o no. Por su forma

de construcción (comparar intersección sobre unión de conjuntos), la relación es simétrica por definición, ya que $A \cup B$ y $A \cap B$ no dependen del orden; esta simetría es lo que permite representarla como un grafo no dirigido y no necesitar aristas con dirección. En cambio, como se verificó en la sección de resultados, la relación no cumple la propiedad de transitividad en general.

Índice de Jaccard: El índice de Jaccard es una medida de similitud entre dos conjuntos A y B , definida como $J(A, B) = \frac{|A \cap B|}{|A \cup B|}$, es decir, el tamaño de la intersección dividido entre el tamaño de la unión; su valor está siempre entre 0 (conjuntos disjuntos) y 1 (conjuntos idénticos). En el proyecto, este índice se calcula por separado para materias, horarios y hobbies, y luego se combinan en una suma ponderada que da la compatibilidad final entre dos estudiantes. Se eligió Jaccard sobre otras métricas porque normaliza el resultado entre 0 y 1 sin importar cuántos elementos tenga cada conjunto, lo que evita que un estudiante con muchas materias parezca más compatible solo por tener conjuntos más grandes.

Grafos: La representación gráfica de los elementos de un conjunto y las relaciones binarias sobre estos se les conoce como grafo y consta de puntos en el espacio, algunos de los cuales están unidos entre sí mediante líneas. Los puntos del grafo se llaman vértices o nodos y representan los elementos del conjunto. Por su parte, las líneas se conocen con el nombre de lados o aristas y representan a aquellos elementos de la forma (i, j) que establecen relación entre los vértices; esto es, los vértices i y j están relacionados. En este proyecto, el conjunto completo de estudiantes y sus relaciones de compatibilidad se representa como un grafo no dirigido, donde cada nodo es un estudiante y existe una arista entre dos nodos si su compatibilidad supera un umbral definido. El uso del grafo permite solucionar el problema de una forma más global, esta estructura permite calcular cosas como qué número de estudiantes son compatibles con uno específico, qué número de conexiones tiene un estudiante específico o si hay grupos pequeños donde todos los estudiantes estén conectados entre sí. Sin el uso del grafo sería mucho más complicado encontrar una forma de hacer estos cálculos.

III. DISEÑO DE LA SOLUCIÓN

Entradas: El programa usa una lista de estudiantes simulados donde cada uno tiene tres atributos representados como conjuntos: materias, horario disponibles y hobbies. Estos datos son simulados y no provienen de estudiantes reales para poder probar el sistema con un volumen y una variedad controlados.

Salidas: El programa produce tres resultados principales: un índice o puntaje de compatibilidad entre cada par de estudiantes calculado como una suma ponderada de los índices de Jaccard de materias, horario y hobbies; un grafo no dirigido donde los estudiantes compatibles quedan conectados; y para cada par conectado, una lista de cosas en común que justifica por qué se recomienda esa conexión.

Restricciones: El umbral de compatibilidad para crear una arista se fijó en un valor base de 0.5 elegido tras comparar el

comportamiento del grafo con 0.3, 0.5, 0.6 y 0.7, sin embargo, está pensado que el estudiante pueda cambiar este índice para ajustar las recomendaciones a sus preferencias.

Componentes y flujo general: El programa tiene cuatro etapas:

1. Carga de la lista de estudiantes: creación y lectura de los datos simulados de cada estudiante.
2. Construcción de conjuntos: a partir de los datos crudos se arma para cada estudiante el conjunto de materias, el de horario y el de hobbies.
3. Cálculo de compatibilidad: se calcula el índice de Jaccard entre cada par de estudiantes para cada atributo, y se combinan en la suma ponderada de compatibilidad.
4. Construcción y análisis del grafo: los pares con compatibilidad mayor al umbral se convierten en aristas de un grafo no dirigido, sobre este grafo se calculan componentes conexas, grado de nodo y la función de cosas en común.

IV. IMPLEMENTACIÓN

El programa está escrito en el lenguaje de Python y usa cuatro librerías principales:

- **Pandas:** Se utilizó para leer los datos del CSV donde están los datos simulados de 20 estudiantes, acceder a sus columnas de materia, horarios y hobbies para crear los conjuntos y hacer el cálculo de las similitudes entre estudiantes, que es la base de toda la lógica del programa. También se usó posteriormente para acceder a la información de los estudiantes y hacer unas consultas simples.
- **NetworkX:** Se utilizó para la creación del grafo donde se representan las relaciones de compatibilidad entre los estudiantes, los nodos representan a cada estudiantes y las aristas son conexiones entre estudiantes que superan un umbral de compatibilidad; también se usó para hacer consultas de las conexiones para saber qué estudiantes son vecinos y hacer recomendaciones en base a esto, se usó una búsqueda de cliques para conocer subgrupos donde todos los estudiantes tiene alguna conexión y se hizo unas consultas sencillas para hacer un análisis básico del grafo sobre el grado de los nodos y sus componentes conexas.
- **Matplotlib:** Se utilizó para poder ver la gráfica del grado y poder entender más fácilmente su estructura y comportamiento.
- **Streamlit:** Se utilizó para crear una pequeña interfaz interactiva donde visualizar el grafo, la lista de estudiantes y las recomendaciones; también para añadir la posibilidad de cambiar el umbral de compatibilidad y ver cómo se comporta el grafo con este cambio.
- Adicionalmente se usó la librería `itertools` de una forma muy secundaria, para generar una lista con pares diferentes y evitar calcular la compatibilidad de dos estudiantes específicos más de una vez.

El desarrollo se hizo primero en Jupyter Notebook, en el archivo `buscacompis.ipynb`, para escribir las funciones de una

forma aislada y poder hacer pruebas de una forma sencilla y evitar la propagación de errores; y luego se migró a un script de Streamlit, en el archivo BuscaCompis.py, con el que se creó la interfaz gráfica utilizando las funciones escritas en el Notebook. Con esta separación es más fácil la verificación de errores y validación de resultados. A continuación, haré una breve explicación de las funciones hechas dividiéndolas en módulos o bloque temático para facilitar la explicación.

Preparación de datos:

- `texto_a_conjunto(texto)`: Esta función convierte un texto en un conjunto de Python usando ; como separador. El propósito de esta función es convertir las columnas de materias, horario y hobbies de los estudiantes en conjuntos con los que se puedan hacer operaciones.
- `cargar_datos()`: Esta función lee el CSV con los estudiantes y agrega a cada estudiante tres nuevas columnas con los conjuntos correspondientes a sus materias, horario y hobbies.

Cálculo de compatibilidad:

- `jaccard(conjunto_a, conjunto_b)`: Esta función calcula el índice de Jaccard entre dos conjuntos. El propósito es tomar dos estudiantes y calcular el índice de Jaccard para sus tres pares de conjuntos, un índice para los conjuntos de materias de los estudiantes a y b, otro índice para los conjuntos de horarios y otro índice para los conjuntos de sus hobbies.
- `suma_ponderada(estudiante_a, estudiante_b, ...)`: Esta función calcula los tres índices mencionados en la función pasada, le asigna un valor a cada índice y hace una suma ponderada de los tres. El propósito de esta función es combinar los tres índices de Jaccard en un solo puntaje de compatibilidad reflejando la importancia que tiene cada apartado, ya que la compatibilidad de materias y horario es más importante que la de hobbies, se asignaron unos pesos de 0.5 0.3 y 0.2 respectivamente.

Construcción y análisis del grafo:

- `construir_grafo(df, umbral)`: Esta función construye el grafo que representa a los estudiantes y sus compatibilidades, se crea un nodo por cada estudiante y se crea una arista entre dos estudiantes solo si su compatibilidad supera el umbral definido. Se fijó un umbral de compatibilidad base de 0.5 pero la idea es que el usuario pueda cambiarlo a su preferencia.
- `recomendar_companeros(G, id_estudiante, top_n)`: Esta función accede a los nodos conectados a un estudiante específico, los ordena basándose en su índice de compatibilidad y devuelve una lista con los tres que tienen un mayor índice.
- `elementos_en_comun(df, id_a, id_b)`: Esta función muestra la intersección de materias, hobbies y horario entre dos estudiantes específicos para poder explicar la razón por la que se recomienda a alguien.
- `encontrar_grupos_estudio(G, tamano_minimo)`: Esta función usa la búsqueda de cliques de NetworkX para encontrar subconjuntos de estudiantes mutuamente compatibles

entre sí, la idea es añadir una opción para encontrar grupos de estudio completos sin tener que estar buscando los estudiantes de uno en uno.

Verificación de propiedades de la relación:

- `verificar_simetria(G)`: Esta función comprueba que si A es compatible con B implica también que B es compatible con A, aunque esto es evidente igual se incluyó para dejarlo escrito en el programa.
- `encontrar_no_transitividad(G, limite_ejemplos)`: Esta función busca ejemplos donde A es compatible con B, B es compatible con C, pero A no es compatible con C; con la idea de mostrar que la relación de compatibilidad no es transitiva en general.

V. PRUEBAS Y RESULTADOS

A continuación se mostrarán las pruebas que se realizaron para verificar si las funciones devuelven los resultados esperados y en caso de no hacerlo, cómo se solucionaron que decisiones se tomaron en base a los resultados obtenidos.

Para verificar el funcionamiento correcto del código que calcula la compatibilidad entre estudiantes se realizó la siguiente prueba:

```
#Probamos la suma ponderada entre varios estudiantes para ver que los números tengan sentido
parejas_prueba = [(0, 1), (0, 5), (0, 14), (0, 19)]
for i, j in parejas_prueba:
    a = df.iloc[i]
    b = df.iloc[j]
    s = suma_ponderada(a, b)
    print(f"{a['nombre']:20s} <-> {b['nombre']:20s} score = {s:.3f}")
```

Ana Torres	<-> Carlos Ruiz	score = 0.350
Ana Torres	<-> Andrés López	score = 0.690
Ana Torres	<-> Gabriela Suárez	score = 1.000
Ana Torres	<-> Felipe Aguilar	score = 0.000

Fig. 1. Prueba de resultados de los cálculos de compatibilidad entre un par de estudiantes.

Los estudiantes fueron simulados de tal manera que el perfil de Ana Torres sea exactamente igual al de Gabriela Suarez y totalmente distinto al de Felipe Aguilar, como se puede ver en la prueba de la función `suma_ponderada` que calcula la similitud entre los tres conjuntos de dos estudiantes: la compatibilidad de Ana Torres y Gabriela Suárez es 1 mientras que la compatibilidad de Ana Torres y Felipe Aguilar es 0. Esto muestra que la parte del código que calcula la compatibilidad entre estudiantes está correcta y se puede trabajar en base a estos cálculos para realizar el resto del programa.

Este cálculo tiene una limitación relacionada con los strings pertenecientes a los datos de los estudiantes:

```
def jaccard(conjunto_a, conjunto_b):
    # Esta función calcula la similitud de Jaccard entre dos conjuntos.
    # Si ambos conjuntos están vacíos, devolvemos 0 para evitar dividir por cero.
    interseccion = conjunto_a & conjunto_b
    union = conjunto_a | conjunto_b
    if len(union) == 0:
        return 0.0
    return len(interseccion) / len(union)

# Prueba con dos conjuntos inventados
prueba_a = ("Cálculo I", "Programación I", "Física I")
prueba_b = ("Cálculo I", "Programación I", "Matemáticas Discretas I")
print("Jaccard de prueba:", jaccard(prueba_a, prueba_b))

Jaccard de prueba: 0.0
```

Fig. 2. Prueba de resultados de los cálculos de compatibilidad donde los strings de los conjuntos no coinciden exactamente.

Como se puede observar, el programa aún no implementa una forma de limpiar los datos o formatear los string, por lo que si hay una diferencia entre los espacios, letras mayúsculas o minúsculas de dos elementos de conjuntos diferentes que representan la misma cosa, el programa no tiene forma de detectarlos y regresa un cálculo de compatibilidad de 0.0, aunque los elementos de los conjuntos puedan representar la misma cosa; esto es un apartado a mejorar a futuro cuando se quiera dar la oportunidad a estudiantes reales de ingresar datos.

El grafo fue definido de tal manera que las conexiones entre los nodos que representan a los estudiantes se conectan con una arista sólo si la compatibilidad entre estos es igual o mayor a un umbral de compatibilidad estipulado, a continuación se muestran los grafos generados dependiendo del umbral estipulado:

Paara un umbral igual a 0.3 se tiene:

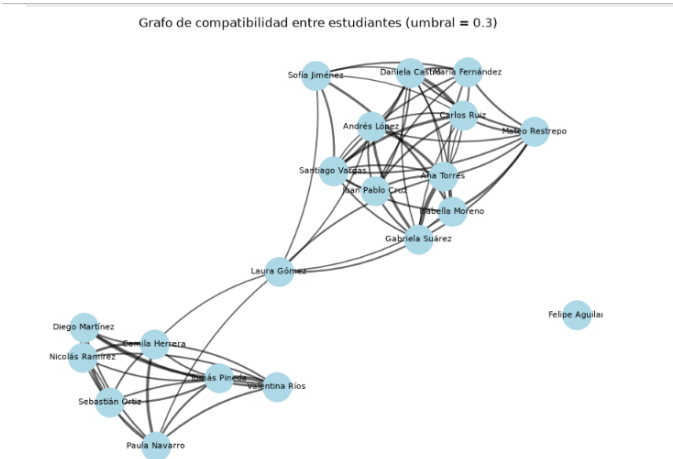


Fig. 3. Grafo generado con un umbral de compatibilidad de 0.3.

Como vemos, un umbral bajo genera un grafo muy conectado, con muchos nodos con grados altos y estudiantes que tienen poca compatibilidad conectados igualmente. Esto podría provocar que las recomendaciones que se hagan sean poco precisas.

Para un umbral igual a 0.7 se tiene:

En cambio, con umbral de 0.7 se genera un grafo muy poco conectado, con muchos nodos aislado y pocas conexiones. Esto podría provocar que el programa no sirva y sea incapaz de realizar recomendaciones.

Finalmente, para un umbral igual a 0.5 se tiene:

Un umbral de 0.5 parece generar un grafo más equilibrado, existen sólo dos estudiante aislados y las conexiones existentes no llegan a ser excesivas.

La tabla anterior corresponde a una comparativa de los grafos generados según el umbral. Como se observa, un umbral por debajo de 0.5 genera un gran número de conexiones y un número bajo de nodos aislados; y por el contrario un umbral alto, por encima de 0.6 genera un número muy bajo de conexiones y un número alto de nodos aislados.

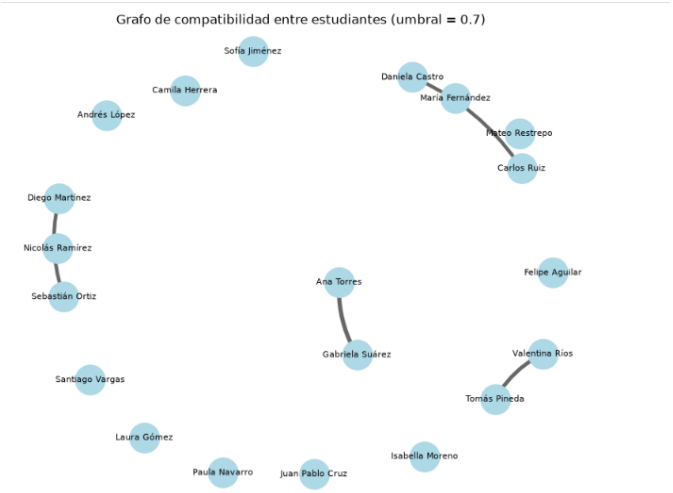


Fig. 4. Grafo generado con un umbral de compatibilidad de 0.7.

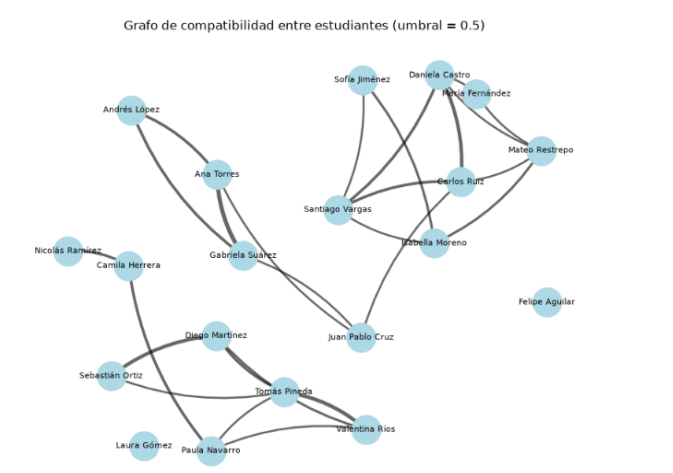


Fig. 5. Grafo generado con un umbral de compatibilidad de 0.5.

TABLA I
COMPARACIÓN DE LAS CARACTERÍSTICAS DE GRAFOS GENERADOS CON DIFERENTES UMBRALES DE COMPATIBILIDAD

Umbral	Nodos	Aristas	Densidad	Estudiantes aislados
0.0	20	190	1.000	0
0.2	20	89	0.468	1
0.3	20	72	0.379	1
0.5	20	26	0.137	2
0.6	20	13	0.068	4
0.7	20	4	0.021	12
0.8	20	4	0.021	12
1.0	20	1	0.005	18

Como unas pruebas de un tema secundario tenemos la verificación de las propiedades de simetría y transitividad que tienen las relaciones binarias que definimos para los pares de estudiantes que cumplen la condición del umbral de compatibilidad:

```
print("¿La relación es simétrica?", verificar_simetria(g))
print("\nEjemplos de no transitividad:")
for a, b, c in encontrar_no_transitividad(g):
    nombre_a = g.nodes[a]["nombre"]
    nombre_b = g.nodes[b]["nombre"]
    nombre_c = g.nodes[c]["nombre"]
    print(f"({nombre_a}) -- ({nombre_b}) -- ({nombre_c}) (pero {nombre_a} y {nombre_c} no son compatibles)")

¿La relación es simétrica? True
Ejemplos de no transitividad:
Andrés López -- Ana Torres -- Juan Pablo Cruz (pero Andrés López y Juan Pablo Cruz no son compatibles)
Santiago Vargas -- Carlos Ruiz -- Juan Pablo Cruz (pero Santiago Vargas y Juan Pablo Cruz no son compatibles)
Santiago Vargas -- Carlos Ruiz -- Mateo Restrepo (pero Santiago Vargas y Mateo Restrepo no son compatibles)
Juan Pablo Cruz -- Carlos Ruiz -- Daniela Castro (pero Juan Pablo Cruz y Daniela Castro no son compatibles)
Juan Pablo Cruz -- Carlos Ruiz -- Mateo Restrepo (pero Juan Pablo Cruz y Mateo Restrepo no son compatibles)
```

Fig. 6. Captura donde se muestran las propiedades de la relación binaria definida para los estudiantes.

Acá se evidencia, como ya suponíamos, que la relación entre un par de estudiantes donde su compatibilidad supera el umbral definido cumple con la propiedad de simetría, si el estudiante A está relacionado con el estudiante B, necesariamente el estudiante B debe estar relacionado con el estudiante A. Por otro lado, se encontraron ejemplos que muestran que la relación no cumple la propiedad de la transitividad, si el estudiante A está relacionado con el estudiante B y B está relacionado con el estudiante C; no significa que necesariamente el estudiante A deba estar relacionado con el estudiante C.

VI. INTERPRETACIÓN DE LOS RESULTADOS.

Los resultados obtenidos permiten distinguir qué aspectos del programa o proyecto funcionan correctamente, qué aspectos funcionan con limitaciones y aspectos que quedan pendientes para resolver en un futuro. Sobre lo que funciona correctamente, la parte matemática del programa correspondiente al cálculo de compatibilidad entre los estudiantes mediante el índice de Jaccard sobre conjuntos, y la construcción del grafo a partir de un umbral funciona correctamente. Las pruebas con perfiles idénticos y perfiles opuestos confirmaron que la función `suma_ponderada` devuelve los valores anormales esperados de 0 para estudiantes que no tienen nada en común y 1 para los perfiles de estudiantes idénticos, el análisis comparativo de umbrales y su importancia en la construcción del grafo mostró un comportamiento consistente y predecible: cuanto más bajo es el umbral hay un mayor número de conexiones y para un umbral alto existe una gran cantidad de nodos aislados. Esto muestra que la parte del programa que está relacionada con los conceptos de matemáticas discreta como son conjuntos, grafos y una mínima parte de relaciones es correcta.

Por otro lado, hay una parte del sistema que funciona solo parcialmente, el cálculo de la similitud entre estudiantes depende de una comparación exacta de strings, no se implementó ninguna forma de normalización y verificación de datos previa. Como se mostró en la sección anterior, dos valores que representan lo mismo como, por ejemplo, la misma materia con una mayúscula o un espacio distinto no son reconocidos

como iguales y el cálculo de compatibilidad los trata como si no tuvieran nada en común. El algoritmo de Jaccard en sí es correcto pero lo que falla es la etapa previa de preparación de los datos, que asume que todos los estudiantes ingresarán la información de forma idéntica.

Adicionalmente, aunque el enfoque de comparar cada par de estudiantes es adecuado para el volumen de datos manejado en este proyecto presenta una limitación de escalabilidad ya que la construcción del grafo requiere calcular la compatibilidad entre todos los pares posibles de estudiantes, lo cual tiene una complejidad de $O(n^2)$ respecto al número de estudiantes n , con los 20 estudiantes usados en las pruebas esto implica 190 comparaciones como se observa en la fila correspondiente al umbral 0.0 de la Tabla I, esta cantidad no es un problema de procesamiento pero si el sistema se aplicara a un volumen de datos mucho mayor, como todos los estudiantes de una facultad o universidad, el número de comparaciones necesarias crecería de forma cuadrática, haciendo que el cálculo del grafo completo se vuelva muy costoso en términos de capacidad de procesamiento necesaria. Esto no es un problema para el alcance actual del proyecto, pero sí en un futuro fuera planeado lanzar una aplicación destinada a un uso real a mayor escala necesitaría pensar en formas de hacer el programa más eficiente, como limitar las comparaciones a estudiantes de la misma carrera o hacer una limitación por ubicación.

Finalmente, hay aspectos que el proyecto todavía no resuelve. El sistema no valida ni corrige la entrada del usuario más allá de exigir que los campos no estén vacíos por lo que la calidad de las recomendaciones depende completamente de qué tan consistentemente escriban los estudiantes reales sus datos. Tampoco existe ningún mecanismo para que el propio sistema sugiera al usuario cuándo dos valores podrían referirse a lo mismo, ni una forma de ajustar el umbral de manera automática según el tamaño o las características del grupo de estudiantes, por ahora solo está implementada la posibilidad de cambiar el umbral manualmente como se hizo en la sección de pruebas.

A modo de resumen, la parte matemática está bien programada y funciona como fue planeado, el objetivo de recomendar a un estudiante posibles compañeros de estudio basándose en su compatibilidad se cumple de cierta forma, pero aún es imposible aplicar este programa en un contexto real debido a la simpleza de la interfaz hecha, la falta de una implementación que limpie los datos, los valide y los guarde de una forma uniforme.

VII. CONCLUSIONES Y TRABAJO FUTURO

El proyecto logró desarrollar una solución funcional para el problema de recomendar compañeros de estudio usando conceptos vistos en matemáticas discretas: conjuntos para representar los atributos de cada estudiante, el índice de Jaccard para calcular similitud entre esos conjuntos, y grafos para modelar y analizar las relaciones de compatibilidad que se crean. Sobre ese grafo fue posible generar recomendaciones individuales para los estudiantes y también identificar grupos de estudio mediante cliques y estudiar propiedades formales

de la relación de compatibilidad como su simetría y su falta de transitividad.

El desarrollo del proyecto me dejó varios aprendizajes importantes. El principal es que los conceptos de matemáticas discretas como conjuntos, relaciones, grafos y combinatoria son herramientas que permiten construir aplicaciones útiles que se pueden plasmar en la realidad para resolver problemas ya que adentro del salón de clase pueden parecer cosas abstractas sin aplicaciones reales. En este proyecto esos conceptos sirvieron como base para crear la estructura para solucionar un problema real cotidiano identificado. También me quedó claro que una relación bien definida no cumple propiedades como la transitividad solo porque el cálculo esté bien hecho, hay que comprobarlo directamente sobre los datos. Otro aprendizaje fue que un algoritmo correcto no es suficiente si los datos que recibe no están bien preparados, el índice de Jaccard funcionó exactamente como estaba pensado pero el problema de los strings mostró que pequeños cambios o inconsistencias en la entrada de datos pueden afectar bastante los resultados. Por último, construir primero la lógica en el notebook y después la interfaz ayudó mucho a identificar más fácilmente si un error venía del cálculo o de la parte visual del programa, lo que hizo que la verificación de errores fuera más rápida y ordenada.

Como trabajo futuro se pueden identificar varias mejoras posibles para una futura versión del programa. La más inmediata es normalizar los datos de entrada eliminando espacios y unificando mayúsculas y minúsculas antes de construir los conjuntos para que el cálculo de compatibilidad no se vea afectado por diferencias de formato provenientes de los inputs del usuario. También estaría bien crear una forma de calcular un umbral adecuado para el grafo dependiendo del tamaño del grupo de estudiantes y ajustarlo de forma automática basado en unos criterios establecidos como número mínimo de conexiones o número máximo de estudiantes aislados, en lugar de depender de que el usuario lo determine manualmente por prueba y error. Otra mejora posible para una siguiente versión sería optimizar el cálculo de compatibilidad para volúmenes de datos más grandes. Actualmente el sistema compara todos los pares posibles de estudiantes, lo que tiene una complejidad de $O(n^2)$ y resultaría poco eficiente si se aplicara a cientos o miles de estudiantes; una posible solución sería limitar las comparaciones a estudiantes de la misma carrera antes de calcular la compatibilidad completa, reduciendo así el número de pares evaluados.

Finalmente, dado que el proyecto cuenta con una interfaz hecha en streamlit bastante básica y limitada, sería bueno mejorarla incluyendo opciones para cargar datos reales de estudiantes mediante un formulario más completo, la posibilidad de crear un perfil individual para cada estudiante, tal vez un sistema de mensajería interna, la posibilidad de implementar una función de detección de proximidad mediante la ubicación del estudiante; en resumen, completar el programa y lograr una versión que pueda distribuirse y ser utilizado por estudiantes reales.

VIII. CONTRIBUCIONES DE LOS INTEGRANTES Y USO DE

Este proyecto fue desarrollado de forma individual por Julian Camilo Hortua Franco encargándose del desarrollo del programa referente al diseño de la solución con la definición de la forma de calcular la compatibilidad y el modelo de grafo, la implementación del cálculo de Jaccard y la construcción del grafo en Python, el desarrollo de la interfaz en Streamlit, las pruebas documentadas en la sección V, y la escritura de este artículo. El historial de commits del repositorio muestra el desarrollo individual y secuencial del proyecto.

En cuanto a la inteligencia artificial, se utilizó como apoyo para la escritura de funciones que requerían el manejo y ordenamiento de diccionarios generados como resultado de funciones de las librerías pandas y networkx, la correcta escritura del código necesario para mostrar los grafos en pantalla y el correcto uso de las funcionalidades de la librería streamlit que se usó para crear la interfaz gráfica del programa. Adicionalmente, el archivo CSV con los estudiantes simulados fue hecho con inteligencia artificialmente de tal forma que los datos permitieran realizar las pruebas documentadas en la sección V referentes al cálculo de compatibilidad para estudiantes idénticos y completamente diferentes, que permitieran además comprobar el incumplimiento de la propiedad de transitividad para la relación binaria entre pares de estudiantes definida.

REFERENCIAS

- [1] M. Guerra Santana, J. Rodríguez Pulido y J. Artilles Rodríguez, "Aprendizaje colaborativo: experiencia innovadora en el alumnado universitario," *REXE. Revista de Estudios y Experiencias en Educación*, vol. 18, no. 36, pp. 269-284, 2019. DOI: 10.21703/rexe.20191836guerra5
- [2] J. F. Villalpando Becerra y A. García Sandoval, *Matemáticas discretas: aplicaciones y ejercicios*. México: Grupo Editorial Patria, 2014.
- [3] Streamlit Inc., "Streamlit documentation," 2024. [En línea]. Disponible en: <https://docs.streamlit.io/>